```
#Customer Transaction Insights and Sanity Checks for Enhanced Retail Analytics

Name : ABHAY CHARAN V
Intern ID : RL26030201
Course : Bachelor Of Technology
Internship Domain : Data Analytics
Submission Date : 14 March 2026
```

```
##Introduction

In today's digital marketplace, business generate large amounts of transactions data from customers
purchases. Ananlyzing this data helps organizations understand customer behaviour, product
demand, and sales trends.

The objective of this project is to analyze customer transaction data to identify important
insights related to product sales, payment methods, discounts, and product returns.
By using data Analytics techniques, we can transform raw data into meaningful information
that supports better business decisions.

In this project, Python and data analytics libraries such as pandas, Matplotlib, and seaborn
are used to clean, process, and visualize the data. Various analytical techniques and visualizations
are applied to understood patterns in customers purchases and product performance.
```

```
## Tools Used

Python
Google colab
Pandas
matplotlib
Seaborn
```

```
## Dateset Description

This project uses two datasets:
1. Customer dataset - contains information about customers
2. Transaction dataset - contains datails about products, purchases, prices, payment methods, and returns.
```

```
## Data Analysis and Visualization
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
customers = pd.read_csv("customer_datasets.csv")
transactions = pd.read_csv("datasets.csv")
```

```
customers.head()
```

	C_ID	Email	Name	Mobile_number	Gender	Age	City	State	Address
0	3768.0	eddie@yahoomail.com	EDDIE	9.045258e+09	M	83.0	Louisville	Kentucky	Masked
1	4852.0	rose@hotmail.com	ROSE	8.834789e+09	F	87.0	Kansas City	Missouri	Masked
2	1174.0	amy@yahoomail.com	AMY	9.557690e+09	F	31.0	Seattle	Washington	Masked
3	4807.0	clarence@gmail.com	CLARENCE	9.394398e+09	M	37.0	Seattle	Washington	Masked
4	9131.0	johnny@hotmail.com	JOHNNY	9.976624e+09	F	80.0	Columbus	Ohio	Masked

Next steps: Generate code with customers New interactive sheet

```
transactions.head()
```

	Credit_card	Product_ID	P_CATEGORY	CONDTION	Brand	Price	Selling_price	Coupon_ID	Date	T
0	4595.0	534	SHOES	New	RKFCM	$1,484.74	$1,471.74	BN710	26/08/2014	17:1
1	6679.0	396	LUGGAGE	New	TNVON	$1,452.50	$1,442.50	KF815	26/01/2014	15:4
2	7075.0	449	SHOES	Refurbished	VKKEA	$4,308.02	$4,284.02	UG960	03/01/2014	1:2
3	1418.0	145	BEDDING	New	PIUEM	$3,272.59	$3,249.59	DK471	30/05/2014	9:1
4	6740.0	844	ELECTRONICS	Used	FMBZC	$628.67	$608.67	OJ311	14/08/2014	5:3

Next steps: Generate code with transactions New interactive sheet

```
customers.shape
```

```
(202, 9)
```

```
transactions.shape
```

```
(9999, 18)
```

```
customers.columns
```

```
Index(['C_ID', 'Email', 'Name', 'Mobile_number', 'Gender', 'Age', 'City ',
       'State', 'Address'],
      dtype='object')
```

```
transactions.columns
```

```
Index(['Credit_card', 'Product_ID', 'P_CATEGORY', 'CONDTION', 'Brand',
       'Price ', 'Selling_price', 'Coupon_ID', 'Date', 'Time', 'GTIN', 'MPN',
       'Merchant_name', 'M_ID', 'Payment Method', 'Transaction ID',
       'Return_ind', 'Return_date'],
      dtype='object')
```

```
customers['Age']
```

	Age
0	83.0
1	87.0
2	31.0
3	37.0
4	80.0
...	...
197	18.0
198	NaN
199	NaN
200	NaN
201	NaN

202 rows × 1 columns

dtype: float64

```
transactions['Selling_price']
```

	Selling_price
0	$1,471.74
1	$1,442.50
2	$4,284.02
3	$3,249.59
4	$608.67
...	...
9994	$1,187.85
9995	$2,202.20
9996	$3,079.80
9997	$1,227.44
9998	$2,883.68

9999 rows × 1 columns

dtype: object

```
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 202 entries, 0 to 201
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   C_ID           198 non-null    float64
 1   Email          198 non-null    object
 2   Name           198 non-null    object
 3   Mobile_number  198 non-null    float64
 4   Gender         198 non-null    object
 5   Age            198 non-null    float64
 6   City           198 non-null    object
 7   State          198 non-null    object
 8   Address        198 non-null    object
dtypes: float64(3), object(6)
memory usage: 14.3+ KB
```

```
transactions.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9999 entries, 0 to 9998
Data columns (total 18 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Credit_card     9994 non-null   float64
 1   Product_ID      9999 non-null   int64
 2   P_CATEGORY      9999 non-null   object
 3   CONDTION        9999 non-null   object
 4   Brand           9999 non-null   object
 5   Price           9999 non-null   object
 6   Selling_price   9999 non-null   object
 7   Coupon_ID       9979 non-null   object
 8   Date            9999 non-null   object
 9   Time            9999 non-null   object
 10  GTIN            9999 non-null   int64
 11  MPN             9999 non-null   int64
 12  Merchant_name   9999 non-null   object
 13  M_ID            9999 non-null   object
 14  Payment Method  9999 non-null   object
 15  Transaction ID  9999 non-null   int64
 16  Return_ind      9999 non-null   int64
 17  Return_date     1532 non-null   object
dtypes: float64(1), int64(5), object(12)
memory usage: 1.4+ MB
```

```
customers.isnull().sum()
```

	0
C_ID	4
Email	4
Name	4
Mobile_number	4
Gender	4
Age	4
City	4
State	4
Address	4

dtype: int64

```
transactions.isnull().sum()
```

	0
Credit_card	5
Product_ID	0
P_CATEGORY	0
CONDTION	0
Brand	0
Price	0
Selling_price	0
Coupon_ID	20
Date	0
Time	0
GTIN	0
MPN	0
Merchant_name	0
M_ID	0
Payment Method	0
Transaction ID	0
Return_ind	0
Return_date	8467

dtype: int64

```
customers['C_ID'].nunique()
```

```
198
```

```
len(customers)
```

```
202
```

```
transactions.describe()
```

	Credit_card	Product_ID	GTIN	MPN	Transaction ID	Return_ind
count	9994.000000	9999.000000	9.999000e+03	9.999000e+03	9999.000000	9999.000000
mean	5728.449470	568.450645	5.459239e+06	5.545180e+08	49977.002100	0.153215
std	2619.421223	253.915314	2.580324e+06	2.585180e+08	28933.097165	0.360213
min	1093.000000	111.000000	1.000909e+06	1.003140e+08	198.000000	0.000000
25%	3500.000000	343.000000	3.176675e+06	3.312793e+08	24419.000000	0.000000
50%	5850.000000	607.000000	5.541936e+06	5.599136e+08	50394.000000	0.000000
75%	7979.000000	772.000000	7.601389e+06	7.765952e+08	74134.000000	0.000000
max	9950.000000	996.000000	9.997547e+06	9.997557e+08	99995.000000	1.000000

```
transactions.sample(5)
```

	Credit_card	Product_ID	P_CATEGORY	CONDTION	Brand	Price	Selling_price	Coupon_ID	Date	
6267	8705.0	970	LUGGAGE	Refurbished	CENQJ	$635.56	$590.56	MB474	27/07/2014	1
8197	8532.0	775	ELECTRONICS	Used	BEFJD	$3,061.88	$3,012.88	RK765	27/07/2014	2
4816	3931.0	470	LUGGAGE	Used	TNVON	$4,783.25	$4,773.25	LT148	21/02/2014	1
7515	2804.0	572	LUGGAGE	New	CENQJ	$3,277.74	$3,247.74	ZQ225	15/11/2014	
6449	2486.0	119	COMPUTERS	New	YXQFM	$901.26	$880.26	QM886	27/05/2014	2

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
customers = pd.read_csv("customer_datasets.csv")
```

```
customers.head()
```

	C_ID	Email	Name	Mobile_number	Gender	Age	City	State	Address
0	3768.0	eddie@yahoomail.com	EDDIE	9.045258e+09	M	83.0	Louisville	Kentucky	Masked
1	4852.0	rose@hotmail.com	ROSE	8.834789e+09	F	87.0	Kansas City	Missouri	Masked
2	1174.0	amy@yahoomail.com	AMY	9.557690e+09	F	31.0	Seattle	Washington	Masked
3	4807.0	clarence@gmail.com	CLARENCE	9.394398e+09	M	37.0	Seattle	Washington	Masked
4	9131.0	johnny@hotmail.com	JOHNNY	9.976624e+09	F	80.0	Columbus	Ohio	Masked

Next steps: Generate code with customers New interactive sheet

```
customers.shape
```

```
(202, 9)
```

```
customers.isnull().sum()
```

	0
C_ID	4
Email	4
Name	4
Mobile_number	4
Gender	4
Age	4
City	4
State	4
Address	4

dtype: int64

```
customers['C_ID'].isnull().sum()
```

```
np.int64(4)
```

```
customers = customers.dropna(subset=['C_ID'])
```

```
customers['C_ID'].isnull().sum()
```

```
np.int64(0)
```

```
customers.shape
```

```
(198, 9)
```

```
print(customers['C_ID'].isnull().sum())
print(customers.shape)
```

```
0
(198, 9)
```

```
customers['C_ID'].nunique()
```

```
198
```

```
len(customers)
```

```
198
```

```
print(customers['C_ID'].nunique())
print(len(customers))
```

```
198
198
```

```
customers.isnull().sum()
```

	0
C_ID	0
Email	0
Name	0
Mobile_number	0
Gender	0
Age	0
City	0
State	0
Address	0

dtype: int64

```
customers.to_csv("clean_customers.csv", index=False)
```

```
!ls
```

```
clean_customers.csv     customer_datasets.csv  sample_data
clean_transactions.csv  datasets.csv
```

```
from google.colab import files
files.download("clean_customers.csv")
```

```
import pandas as pd
```

```
customers = pd.read_csv("customer_datasets.csv")
transactions = pd.read_csv("datasets.csv")
```

```
transactions.head()
```

	Credit_card	Product_ID	P_CATEGORY	CONDTION	Brand	Price	Selling_price	Coupon_ID	Date	T
0	4595.0	534	SHOES	New	RKFCM	$1,484.74	$1,471.74	BN710	26/08/2014	17:1
1	6679.0	396	LUGGAGE	New	TNVON	$1,452.50	$1,442.50	KF815	26/01/2014	15:4
2	7075.0	449	SHOES	Refurbished	VKKEA	$4,308.02	$4,284.02	UG960	03/01/2014	1:2
3	1418.0	145	BEDDING	New	PIUEM	$3,272.59	$3,249.59	DK471	30/05/2014	9:1
4	6740.0	844	ELECTRONICS	Used	FMBZC	$628.67	$608.67	OJ311	14/08/2014	5:3

Next steps: Generate code with transactions New interactive sheet

```
transactions.shape
```

```
(9999, 18)
```

```
transactions.isnull().sum()
```

	0
Credit_card	5
Product_ID	0
P_CATEGORY	0
CONDTION	0
Brand	0
Price	0
Selling_price	0
Coupon_ID	20
Date	0
Time	0
GTIN	0
MPN	0
Merchant_name	0
M_ID	0
Payment Method	0
Transaction ID	0
Return_ind	0
Return_date	8467

dtype: int64

```
transactions['Credit_card'] = transactions['Credit_card'].fillna("Unknown")
```

```
"No Coupon"
```

```
'No Coupon'
```

```
transactions['Coupon_ID'] = transactions['Coupon_ID'].fillna("No Coupon")
```

```
"No Return"
```

```
'No Return'
```

```
transactions['Return_date'] = transactions['Return_date'].fillna("No Return")
```

```
transactions.isnull().sum()
```

	0
Credit_card	0
Product_ID	0
P_CATEGORY	0
CONDTION	0
Brand	0
Price	0
Selling_price	0
Coupon_ID	0
Date	0
Time	0
GTIN	0
MPN	0
Merchant_name	0
M_ID	0
Payment Method	0
Transaction ID	0
Return_ind	0
Return_date	0

dtype: int64

```
transactions.groupby('Product_ID')['Selling_price'].sum()
```

Product_ID	Selling_price
111	$2,409.24 $4,218.22 $2,412.24 $4,232.22 $2,406...
119	$3,103.29 $3,101.29 $886.26 $3,094.29 $870.26 ...
132	$4,498.66 $4,490.66 $2,430.48 $2,430.48 $4,489...
136	$3,984.79 $4,007.79 $3,970.79 $4,006.79 $4,233...
138	$3,499.43 $3,503.43 $3,472.43 $3,956.70 $3,931...
...	...
960	$3,994.10 $3,978.10 $2,849.99 $3,979.10 $3,965...
970	$369.88 $545.56 $371.88 $361.88 $587.56 $606.5...
986	$3,115.26 $3,179.26 $3,187.26 $4,286.83 $4,274...
988	$846.57 $839.57 $837.57 $828.57 $2,051.74 $828...
996	$887.56 $667.38 $663.38 $887.56 $663.38 $689.3...

96 rows × 1 columns

dtype: object

```
transactions.groupby('Product_ID')['Selling_price'].sum().sort_values(ascending=False).head()
```

Product_ID	Selling_price
832	$974.49 $1,003.49 $986.49 $3,729.47 $3,726.47 ...
527	$94.43 $3,297.99 $3,298.99 $97.43 $3,291.99 $9...
996	$887.56 $667.38 $663.38 $887.56 $663.38 $689.3...
988	$846.57 $839.57 $837.57 $828.57 $2,051.74 $828...
141	$802.05 $3,550.93 $3,542.93 $819.05 $816.05 $3...

dtype: object

```
transactions.groupby('Payment Method')['Selling_price'].sum()
```

Payment Method	Selling_price
Credit card	$4,284.02 $4,891.58 $4,764.25 $2,098.08 $4,172...
Debit card	$608.67 $974.49 $846.57 $4,633.93 $1,413.77 $3...
Deirect debits	$3,541.77 $2,879.20 $2,412.48 $359.59 $3,246.1...
Gift card	$3,665.96 $335.54 $2,399.11 $1,174.85 $3,532.7...
Mobile carrier Billing	$1,442.50 $3,698.97 $1,001.90 $3,918.19 $2,383...
Paypal wallet	$1,025.81 $3,605.77 $2,190.16 $4,006.79 $2,002...
Prepaid card	$1,471.74 $3,249.59 $2,802.51 $3,783.74 $289.4...

dtype: object

```
transactions.groupby('P_CATEGORY')['Selling_price'].sum()
```

P_CATEGORY	Selling_price
APPLIANCES	$4,891.58 $2,085.85 $2,109.85 $2,017.24 $2,028...
BABY CLOTHING	$3,918.19 $4,633.93 $3,297.99 $4,609.93 $2,009...
BABY TOYS	$289.49 $3,499.43 $334.28 $1,598.58 $3,093.36 ...
BEDDING	$3,249.59 $1,955.83 $3,710.01 $3,527.28 $1,957...
CLOTHING	$3,704.69 $2,181.20 $887.56 $358.59 $1,025.81 ...
COMPUTERS	$2,098.08 $2,827.97 $846.57 $2,435.58 $2,836.9...
DECOR	$974.49 $4,943.44 $2,636.83 $4,932.73 $390.77 ...
ELECTRONICS	$608.67 $2,802.51 $4,172.69 $3,607.22 $2,784.5...
GAMES	$2,383.99 $2,403.99 $711.39 $4,134.59 $2,446.8...
KITCHEN & DINING	$650.82 $3,984.79 $4,007.79 $4,171.27 $94.43 $...
LUGGAGE	$1,442.50 $3,698.97 $4,764.25 $1,439.50 $376.7...
OFFICE SUPPLIES	$1,001.90 $2,167.71 $2,490.31 $4,652.60 $3,103...
SHOES	$1,471.74 $4,284.02 $3,434.72 $1,257.44 $2,262...

dtype: object

```
transactions.groupby('Product_ID')['Selling_price'].sum().sort_values(ascending=False).head()
```

Product_ID	Selling_price
832	$974.49 $1,003.49 $986.49 $3,729.47 $3,726.47 ...
527	$94.43 $3,297.99 $3,298.99 $97.43 $3,291.99 $9...
996	$887.56 $667.38 $663.38 $887.56 $663.38 $689.3...
988	$846.57 $839.57 $837.57 $828.57 $2,051.74 $828...
141	$802.05 $3,550.93 $3,542.93 $819.05 $816.05 $3...

dtype: object

```
transactions.groupby('Payment Method')['Selling_price'].sum()
```

Payment Method	Selling_price
Credit card	$4,284.02 $4,891.58 $4,764.25 $2,098.08 $4,172...
Debit card	$608.67 $974.49 $846.57 $4,633.93 $1,413.77 $3...
Deirect debits	$3,541.77 $2,879.20 $2,412.48 $359.59 $3,246.1...
Gift card	$3,665.96 $335.54 $2,399.11 $1,174.85 $3,532.7...
Mobile carrier Billing	$1,442.50 $3,698.97 $1,001.90 $3,918.19 $2,383...
Paypal wallet	$1,025.81 $3,605.77 $2,190.16 $4,006.79 $2,002...
Prepaid card	$1,471.74 $3,249.59 $2,802.51 $3,783.74 $289.4...

dtype: object

```
transactions.groupby('P_CATEGORY')['Selling_price'].sum()
```

P_CATEGORY	Selling_price
APPLIANCES	$4,891.58 $2,085.85 $2,109.85 $2,017.24 $2,028...
BABY CLOTHING	$3,918.19 $4,633.93 $3,297.99 $4,609.93 $2,009...
BABY TOYS	$289.49 $3,499.43 $334.28 $1,598.58 $3,093.36 ...
BEDDING	$3,249.59 $1,955.83 $3,710.01 $3,527.28 $1,957...
CLOTHING	$3,704.69 $2,181.20 $887.56 $358.59 $1,025.81 ...
COMPUTERS	$2,098.08 $2,827.97 $846.57 $2,435.58 $2,836.9...
DECOR	$974.49 $4,943.44 $2,636.83 $4,932.73 $390.77 ...
ELECTRONICS	$608.67 $2,802.51 $4,172.69 $3,607.22 $2,784.5...
GAMES	$2,383.99 $2,403.99 $711.39 $4,134.59 $2,446.8...
KITCHEN & DINING	$650.82 $3,984.79 $4,007.79 $4,171.27 $94.43 $...
LUGGAGE	$1,442.50 $3,698.97 $4,764.25 $1,439.50 $376.7...
OFFICE SUPPLIES	$1,001.90 $2,167.71 $2,490.31 $4,652.60 $3,103...
SHOES	$1,471.74 $4,284.02 $3,434.72 $1,257.44 $2,262...

dtype: object

```
transactions['Selling_price'] = transactions['Selling_price'].replace('[\$,]','', regex=True)
```

```
<>:1: SyntaxWarning: invalid escape sequence '\$'
<>:1: SyntaxWarning: invalid escape sequence '\$'
/tmp/ipykernel_840/1705751412.py:1: SyntaxWarning: invalid escape sequence '\$'
  transactions['Selling_price'] = transactions['Selling_price'].replace('[\$,]','', regex=True)
```

```
transactions['Selling_price'] = transactions['Selling_price'].replace(r'\((.*?)\)', r'-\1', regex=True)
```

```
transactions['Selling_price'] = transactions['Selling_price'].astype(float)
```

```
transactions['Selling_price'].dtype
```

```
dtype('float64')
```

```
transactions['Selling_price'] = transactions['Selling_price'].replace('[\$,]', '', regex=True)
transactions['Selling_price'] = transactions['Selling_price'].replace(r'\((.*?))\)', regex=True)
transactions['Selling_price'] = transactions['Selling_price'].astype(float)
```

```
<>:1: SyntaxWarning: invalid escape sequence '\$'
<>:1: SyntaxWarning: invalid escape sequence '\$'
/tmp/ipykernel_840/3957242945.py:1: SyntaxWarning: invalid escape sequence '\$'
  transactions['Selling_price'] = transactions['Selling_price'].replace('[\$,]', '', regex=True)
```

```
transactions['Selling_price'].dtype
```

```
dtype('float64')
```

```
transactions.groupby('Product_ID')['Selling_price'].sum().sort_values(ascending=False).head()
```

Product_ID	Selling_price
694	535859.46
702	527932.96
722	471151.33
604	451495.17
266	435323.00

dtype: float64

```
transactions.groupby('Payment Method')['Selling_price'].sum()
```

Payment Method	Selling_price
Credit card	9526042.90
Debit card	831404.75
Deirect debits	153502.37
Gift card	182324.68
Mobile carrier Billing	10852319.54
Paypal wallet	223567.96
Prepaid card	4500341.97

dtype: float64

```
transactions.groupby('P_CATEGORY')['Selling_price'].sum().sort_values(ascending=False)
```

P_CATEGORY	Selling_price
DECOR	3200896.29
COMPUTERS	3061094.82
ELECTRONICS	2685133.42
OFFICE SUPPLIES	2466129.19
SHOES	2298163.27
KITCHEN & DINING	2017689.71
CLOTHING	1972876.86
BEDDING	1661701.45
GAMES	1515700.09
APPLIANCES	1510925.54
LUGGAGE	1422861.12
BABY CLOTHING	1342503.30
BABY TOYS	1113829.11

dtype: float64

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
transactions.columns
```

```
Index(['Credit_card', 'Product_ID', 'P_CATEGORY', 'CONDTION', 'Brand',
       'Price ', 'Selling_price', 'Coupon_ID', 'Date', 'Time', 'GTIN', 'MPN',
       'Merchant_name', 'M_ID', 'Payment Method', 'Transaction ID',
       'Return_ind', 'Return_date'],
      dtype='object')
```

```
transactions.columns = transactions.columns.str.strip()
```

```
transactions['Price'] = transactions['Price'].replace('[\$,]', '', regex=True)
```

```
<>:1: SyntaxWarning: invalid escape sequence '\$'
<>:1: SyntaxWarning: invalid escape sequence '\$'
/tmp/ipykernel_840/239911847.py:1: SyntaxWarning: invalid escape sequence '\$'
  transactions['Price'] = transactions['Price'].replace('[\$,]', '', regex=True)
```

```
transactions['Price'] = transactions['Price'].replace(r'\((.*?)\)', r'-\1', regex=True)
```

```
transactions['Price']= transactions['Price'].astype(float)
```

```
transactions['Discount'] = transactions['Price'] - transactions['Selling_price']
```

```
transactions.groupby('Payment Method')['Discount'].sum().sort_values(ascending=False)
```

Payment Method	Discount
Mobile carrier Billing	123437.0
Credit card	112146.0
Prepaid card	47189.0
Debit card	9178.0
Paypal wallet	2471.0
Deirect debits	2429.0
Gift card	1777.0

dtype: float64

```
transactions['Price'].dtype
```

```
dtype('float64')
```

```
transactions['Return_ind'].value_counts()
```

Return_ind	count
0	8467
1	1532

dtype: int64

```
transactions.groupby('P_CATEGORY')['Return_ind'].sum().sort_values(ascending=False).head()
```

P_CATEGORY	Return_ind
DECOR	182
COMPUTERS	171
ELECTRONICS	159
SHOES	141
OFFICE SUPPLIES	122

dtype: int64

```
transactions.groupby('Merchant_name')['Discount'].sum().sort_values(ascending=False).head()
```

Merchant_name	Discount
Barnes & Noble	23173.0
L'Occitane	21734.0
Ulta	20607.0
Guitar Center	18886.0
DODOcase	18810.0

dtype: float64

```
transactions['CONDTION'].value_counts()
```

CONDTION	count
New	4625
Used	2903
Refurbished	2471

dtype: int64

```
transactions.to_csv('clean_transactions.csv', index=False)
```

```
transactions.groupby('Merchant_name')['Discount'].sum().sort_values(ascending=False).head()
```

Merchant_name	Discount
Barnes & Noble	23173.0
L'Occitane	21734.0
Ulta	20607.0
Guitar Center	18886.0
DODOcase	18810.0

dtype: float64

```
transactions.to_csv('clean_transactions.csv', index=False)
```

```
from google.colab import files
files.download('clean_transactions.csv')
```

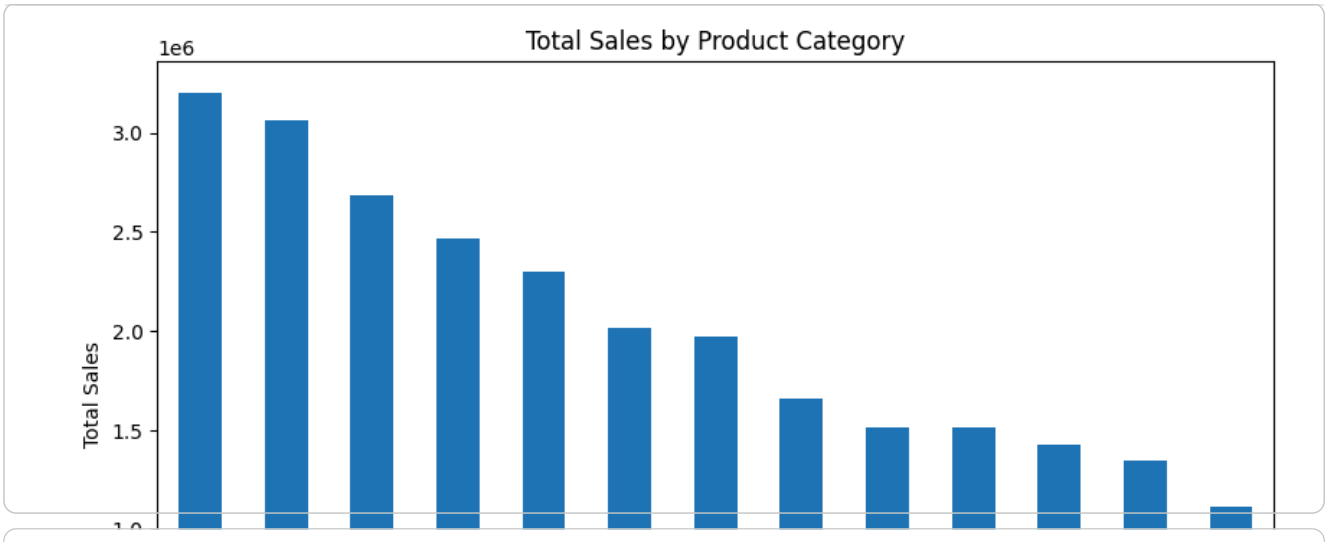
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
transactions = pd.read_csv("clean_transactions.csv")
```

```
transactions['Discount'] = transactions['Price'] - transactions['Selling_price']
```

```
category_sales = transactions.groupby('P_CATEGORY')['Selling_price'].sum().sort_values(ascending=False)
```

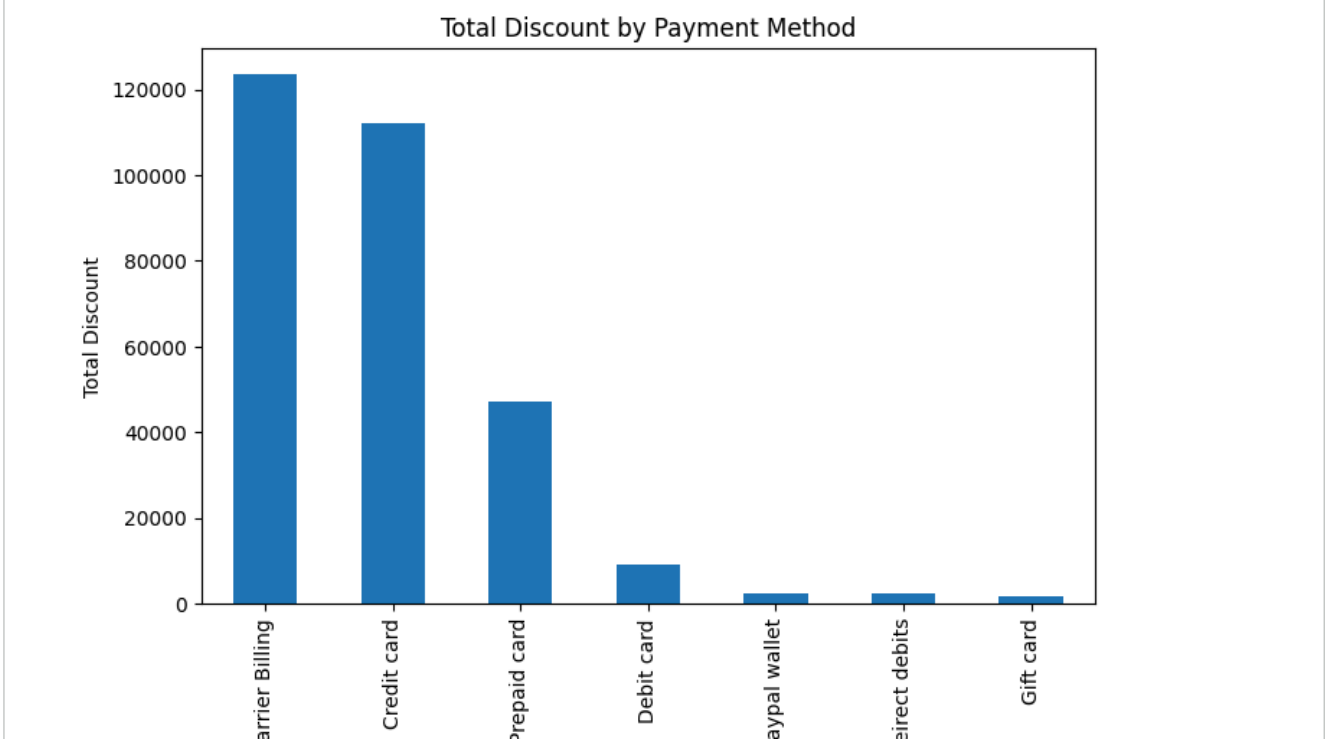
```
plt.figure(figsize=(10,6))
category_sales.plot(kind='bar')
plt.title('Total Sales by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Total Sales')
plt.show()
```

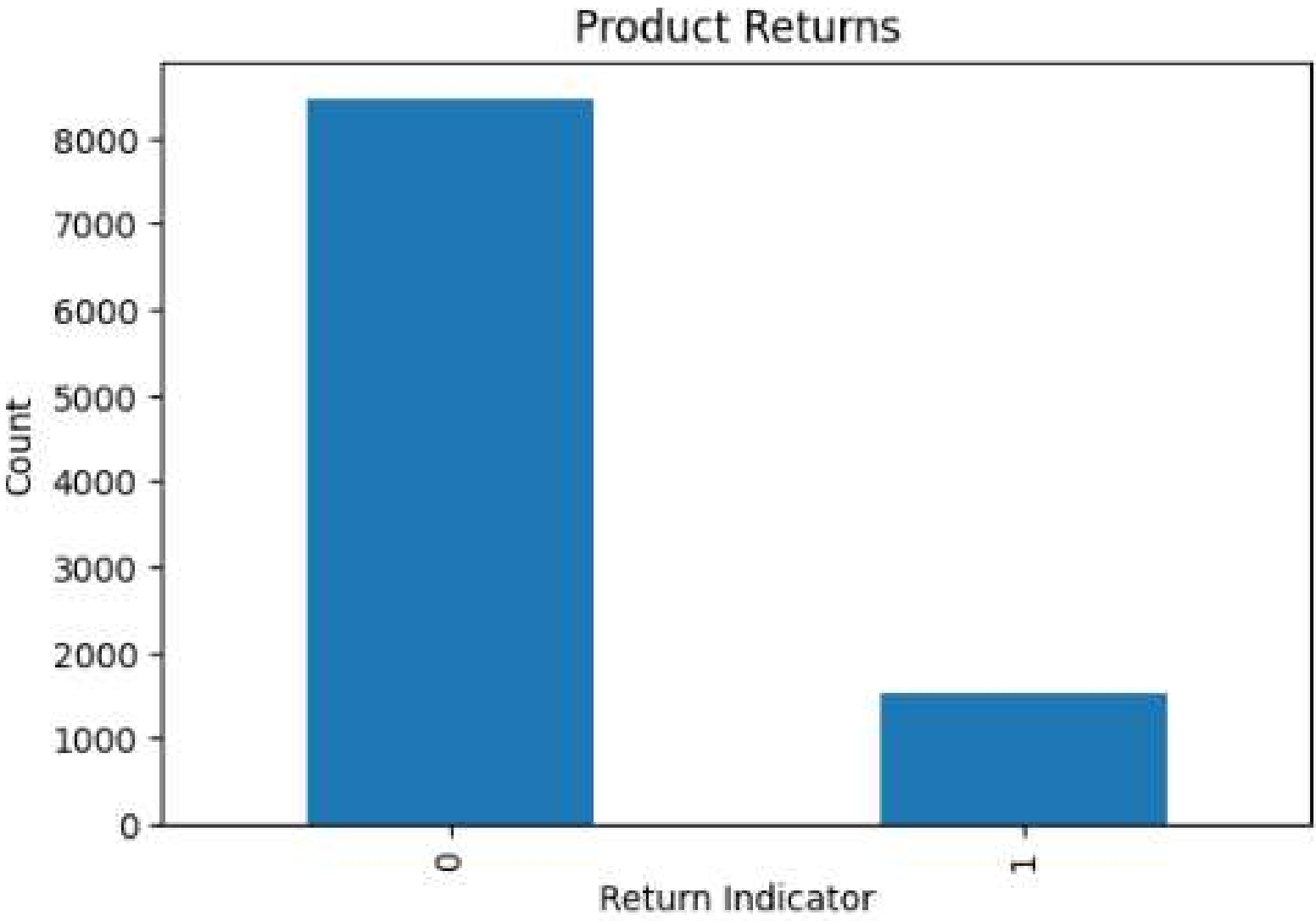


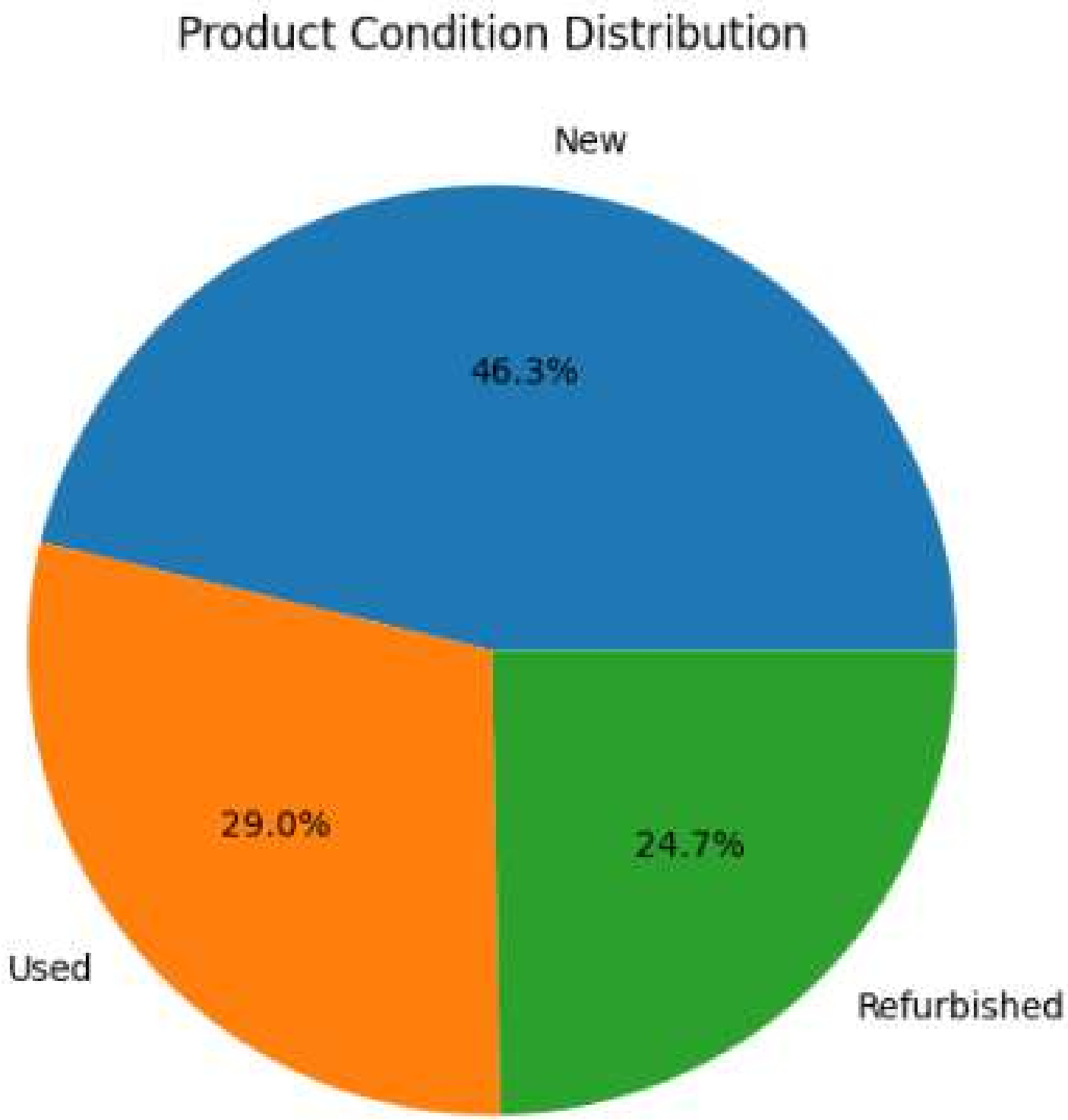

```
Insight

The analysis shows that the DECOR category generated the highest
total sales among all product categories.
This indicates that consumers/customers are highly
interested in decorative products compared to other categories.
```

```
payment_discount = transactions.groupby('Payment Method')['Discount'].sum().sort_values(ascending=False)
```

```
plt.figure(figsize=(8,5))
payment_discount.plot(kind='bar')
plt.title('Total Discount by Payment Method')
plt.xlabel('Payment Method')
plt.ylabel('Total Discount')
plt.show()
```




Product Returns
Count
8000
7000
6000
5000
4000
3000
2000
1000
0
0
1
Return Indicator


Product Condition Distribution
New
46.3%
29.0%
24.7%
Used
Refurbished